PROJECT REPORT

ROVER

Multi-Mode Arduino car with Infrared Remote Control, Obstacle Avoidance, and Line Following, Extended with Voice Control, Object Recognition and Shared-Control Autonomy

Arduino · Python · Computer Vision · Object Detection · Voice Interface · Bluetooth Low Energy

Submitted by

Mihir Bhati

Ahana Sharan

Archit Khandelwal

Arnav Astu

Course	Embedded Systems (BCSE305L)
Department	B.Tech Computer Science and Engineering (Core)

October 2026

Abstract

This project presents **ROVER**, a low-cost four-wheel robot car that can be driven by an infrared remote, by natural voice commands, by typed routes and through an immersive full-screen “VR View”, and that can carry out simple autonomous tasks while a human supervises it. The car is built around an Arduino Uno R3, an L298N motor driver, an HC-SR04 ultrasonic sensor on a servo, line sensors and an HC-08 Bluetooth Low Energy module. A smartphone mounted on the car streams video over Wi-Fi, and a laptop running Python performs all computation: image analysis, object detection, voice-command understanding, route planning and safety supervision.

Obstacles are judged by combining a transparent image-processing method (floor-colour continuity) with real ultrasonic distance, and everyday objects are recognised with a pre-trained YOLOv8-nano neural network executed through ONNX Runtime. On top of manual and voice driving, the system provides obstacle bypass to a target point, an autopilot that accepts the operator’s spoken corrections in real time, teach-and-repeat routes, return-to-home, object finding, person following and spoken scene descriptions. Safety is designed into every layer: every movement the laptop requests is a pulse of at most 350 ms that the Arduino ends by itself, so a lost connection can never leave the car driving.

The software is covered by 97 automated tests, including simulations of the car and its sensors; real-world operation is documented with photographs, logs and measurements. The report also states the system’s limitations honestly — for example, that distances are estimated from timing, and that the pre-trained detector does not recognise a steel flask as a bottle.

Keywords: Arduino, robot car, computer vision, YOLOv8, ONNX Runtime, speech recognition, Bluetooth Low Energy, shared control, teach-and-repeat, human-in-the-loop.

Contents

1 Introduction 4

1.1 Background 4

1.2 Problem statement 4

1.3 Objectives 4

1.4 Scope 4

2 System Architecture 5

2.1 Software layers 5

2.2 How one command flows through the system 6

3 Hardware Design 7

3.1 Working principles 7

4 Technologies Used 9

4.1 Why these choices 10

5 Firmware and Communication 11

5.1 Bounded pulses 11

5.2 Bluetooth Low Energy link 11

5.3 Camera link 12

6 Features — Working, Technology and Use Cases 13

6.1 Remote-control modes (original car) 14

6.2 Web dashboard and manual driving 14

6.3 Voice control 15

6.4 Routes by text or voice 16

6.5 Camera floor analysis 16

6.6 Ultrasonic safety gates 17

6.7 Pass obstacle 17

6.8 VR View — immersive driving interface 17

6.9 Speed levels 1–10 18

6.10 Analyze and report 19

6.11 Autopilot with operator corrections (shared control) 19

6.12 Teach-and-repeat and Go home 19

6.13 Go to object 20

6.14 Follow me 20

6.15 What do you see — spoken scene description 20
6.16 Safety architecture 21
7 Computer Vision and Object Detection 22
7.1 Floor-colour continuity (image processing) 22
7.2 Object detection with YOLOv8 23
7.3 Depth estimation experiment 25
7.4 Autopilot decision logic 25
8 Motion Model and Calibration 27
9 Testing and Results 28
9.1 Automated tests 28
9.2 Simulated autopilot result 28
9.3 Real-world evidence 28
9.4 Problems found and solved 29
10 Limitations and Future Work 30
10.1 Training plan for a custom detector 30
11 Conclusion 31
References 32
Appendix A — Voice Command Reference 33
Appendix B — Project Files and Running the System 34

1 Introduction

1.1 Background

Small mobile robots are now common in homes (robot vacuum cleaners), warehouses (automated guided vehicles) and inspection work. Most commercial systems are closed: their behaviour cannot be inspected or changed, and they rely on expensive sensors such as lidar. At the same time, almost everyone carries a smartphone with a good camera and a laptop with enough computing power to run modern vision and speech software. This project explores how far a very low-cost robot car can go when the phone is used as its eyes and the laptop as its brain.

1.2 Problem statement

To design and build a robot car that a person can control naturally (by voice, and through an immersive camera view), that can perceive obstacles and everyday objects, that can perform simple autonomous tasks while the operator stays in control, and that remains safe even when communication fails — using only inexpensive hardware.

1.3 Objectives

- Keep the original Arduino car modes (remote control, obstacle avoidance, line tracking) working unchanged.
- Provide manual, typed-route and voice control through a local web dashboard.
- Detect obstacles by fusing camera image analysis with ultrasonic distance.
- Recognise everyday objects with a neural network and use them for “go to”, “follow me” and scene description.
- Provide an immersive VR-style interface with directional danger warnings, a speed gauge and a path map.
- Implement shared-control autonomy: an autopilot, teach-and-repeat and return-to-home that accept voice corrections.
- Make safety a design property: bounded movements, emergency stop, heartbeats and fault isolation.
- Verify the software with automated tests and document real-world results honestly.

1.4 Scope

The project covers indoor operation on a flat floor. Positions and distances travelled are estimated from motor timing (no wheel encoders or compass are fitted). No neural network was trained for this project; pre-trained models were used, and a training plan is given as future work.

2 System Architecture

ROVER is divided between three devices. The **phone** provides vision, the **laptop** provides all computation and the user interface, and the **Arduino** controls motors and sensors. Images never pass through the Arduino; it only receives very short text commands.

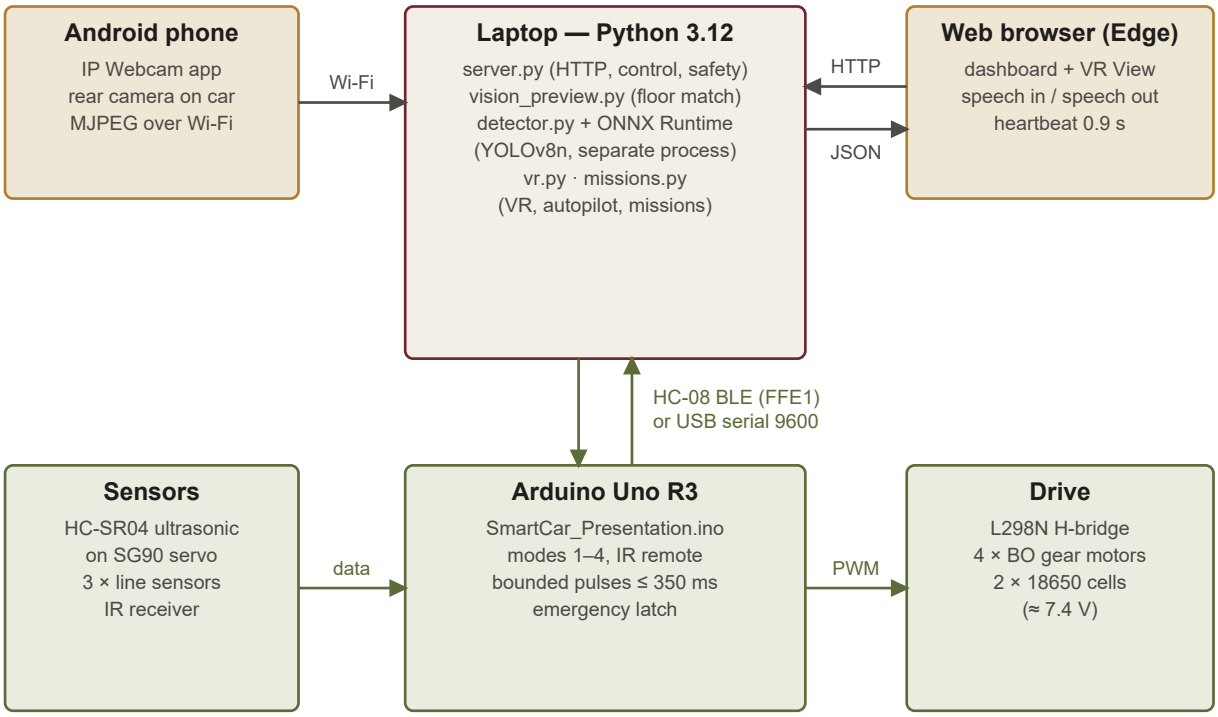


Figure 2.1. *Block diagram of the complete system and the communication links between its parts.*

2.1 Software layers

The software is organised in five layers. Each layer only talks to the one next to it, which keeps the safety logic in one place and made it possible to add new features (VR View, missions) as separate modules without changing the others.

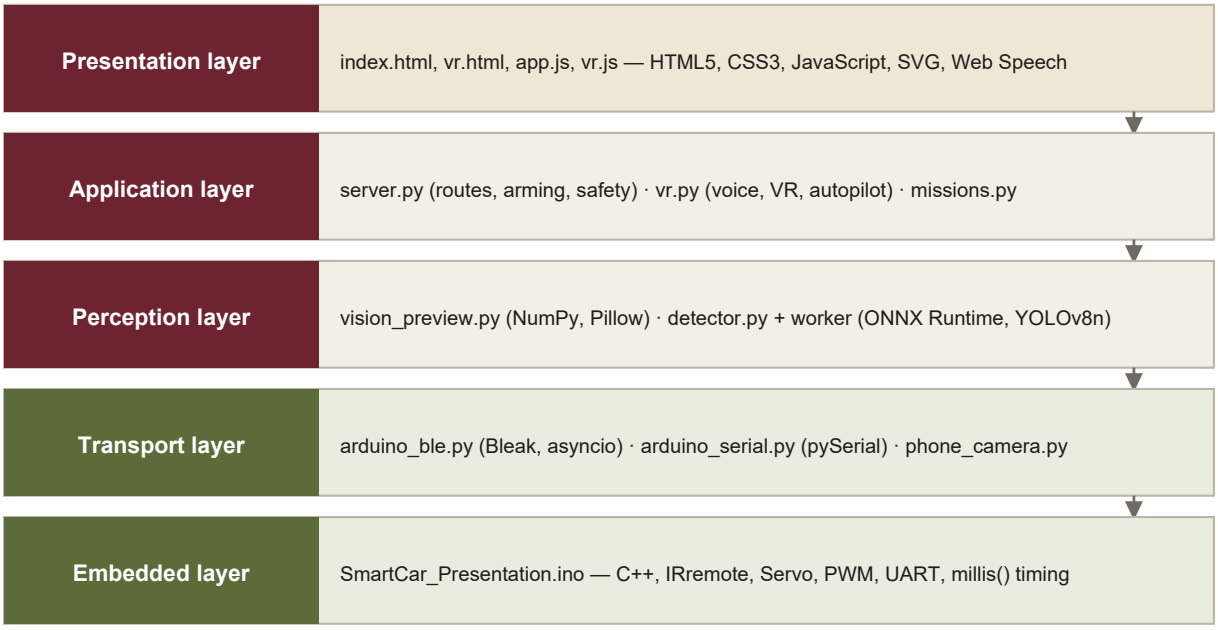


Figure 2.2. Layered software architecture with the main files and technologies of each layer.

2.2 How one command flows through the system

The sequence below follows a single spoken command from the operator to the motors and back. Every movement is confirmed twice by the Arduino: once when it starts (*ACK*) and once when it has ended by itself (*MOVE_DONE*).

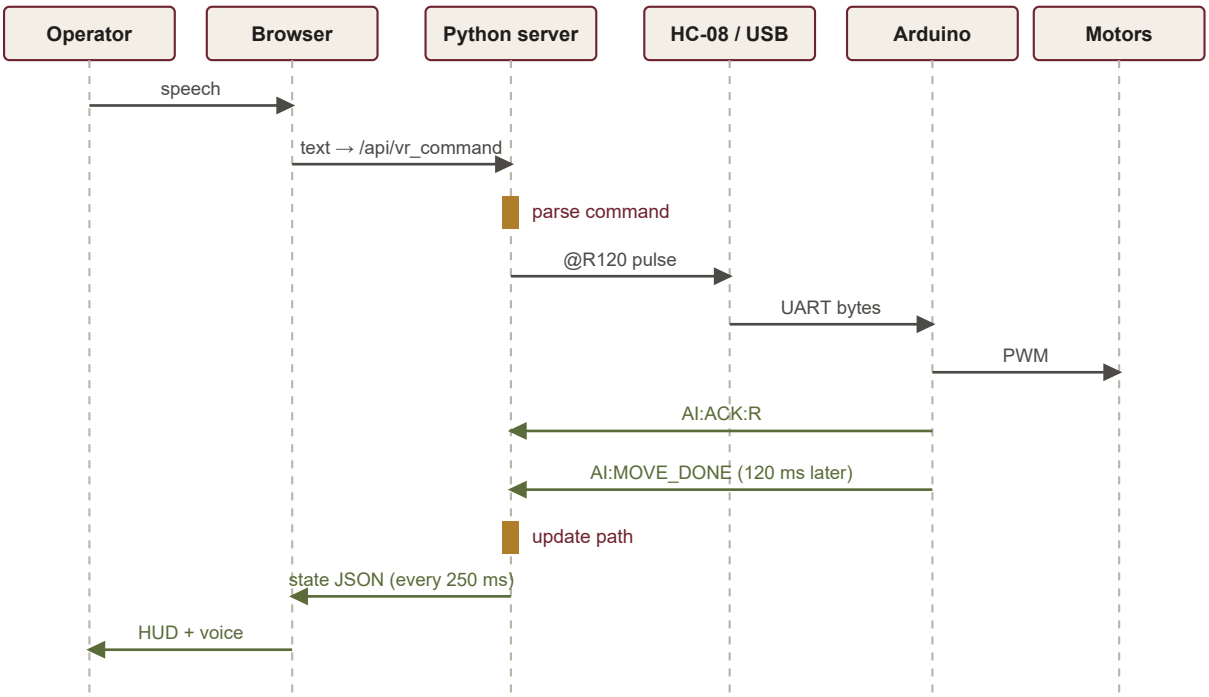


Figure 2.3. Sequence diagram for the voice command "turn right 30 degrees".

3 Hardware Design

Table 3.1. Hardware components.

Component	Specification	Role in ROVER
Arduino Uno R3	ATmega328P, 16 MHz, 32 KB flash, 2 KB RAM	Main controller: motors, sensors, original modes, command protocol
Sensor Shield v5	3-pin signal/5 V/GND headers	Tidy connections for sensors and servo
L298N driver	Dual H-bridge, PWM enable inputs	Switches battery power to the motors; sets speed and direction
4 × BO gear motors	DC, skid-steer layout	Drive; turning by running the two sides in opposite directions
2 × 18650 Li-ion cells	≈ 7.4 V nominal	Power for motors and electronics
HC-SR04 ultrasonic sensor	Range ≈ 3–300 cm, beam ≈ ±15°	Distance measurement (the only distance source)
SG90 micro servo	0–180°	Turns the ultrasonic sensor left (170°), centre (90°), right (10°)
3 × IR line sensors	Digital outputs	Line-tracking mode
IR receiver + remote	NEC protocol	Arrows drive, OK stops, keys 1–4 select modes
HC-08 BLE module	Bluetooth 4.0, UART	Wireless link to the laptop (service FFE0, characteristic FFE1)
Android smartphone	IP Webcam app, 1080 × 1920	Car-mounted camera streaming over Wi-Fi
Laptop	Windows 11, Python 3.12	All computation and the user interface

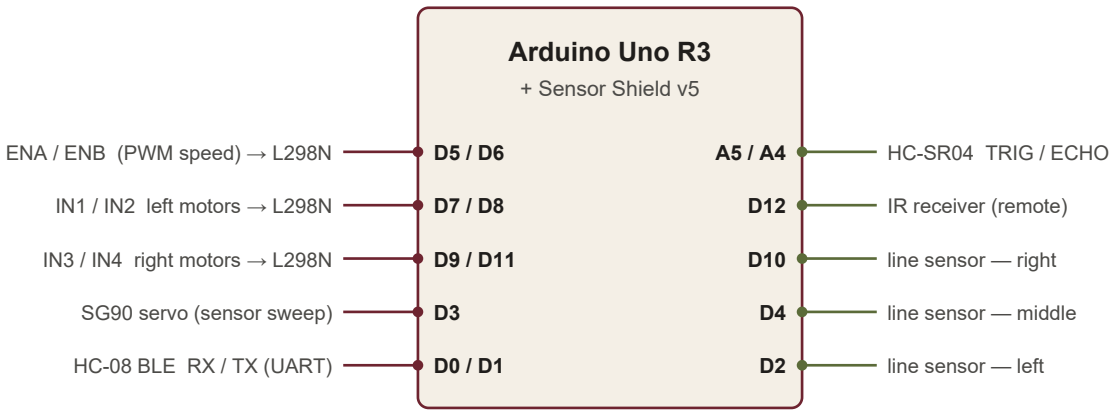


Figure 3.1. *Pin assignments on the Arduino (taken from the firmware source).*

3.1 Working principles

Ultrasonic distance

The HC-SR04 sends a 40 kHz sound burst and measures the time until the echo returns. Sound travels about 343 m/s, and the pulse goes to the object and back, so **distance (cm) = echo time (µs)**

÷ **58**. The firmware treats a missing echo or a value outside 3–300 cm as invalid and never as "clear".

Speed and direction (PWM and H-bridge)

The L298N contains two H-bridges — four switches per motor pair that can reverse the current, and so the direction of rotation. Speed is set by **pulse-width modulation**: the enable pin is switched on and off very quickly and the fraction of "on" time sets the average voltage. ROVER uses `analogWrite(ENA, 200)`, i.e. about 78 % power.

Power

The motors are powered only from the 18650 cells. During development the car stopped moving although the Arduino lit up from USB power; a motor-only test program proved the cause was a discharged battery, not the software (section 9.4).

4 Technologies Used

Table 4.1 lists every technology used in the project, its version and where it is used. Versions were read from the installed packages on the development laptop.

Table 4.1. Complete list of technologies, versions and their purpose.

Technology	Version	Where and why it is used
C++ (Arduino)	—	Firmware SmartCar_Presentation.ino (761 lines); compiled to machine code for the Uno
Arduino IDE / arduino-cli	2.3.10 / 1.5.1	Compiling and uploading; the CLI allows scripted, repeatable builds
Arduino AVR core (avr-gcc, avrdude)	1.8.8	Compiler and uploader for the ATmega328P
IRremote library	4.7.1	Decoding remote-control codes (e.g. UP = 0xB946FF00)
Servo library	bundled	Driving the SG90 that sweeps the ultrasonic sensor
Python	3.12.14	All laptop software
http.server, threading, asyncio, re, json	standard library	Local web server, background jobs, BLE event loop, voice grammar, API messages
NumPy	2.3.5	Vectorised pixel maths for image analysis and detection post-processing
Pillow (PIL)	12.3.0	Decoding, rotating, cropping, resizing frames; drawing analysis boxes
pySerial	3.5	USB serial link to the Arduino (9600 baud)
Bleak + WinRT	3.0.2 / 3.2.1	Bluetooth Low Energy link to the HC-08 without Windows pairing
ONNX Runtime	1.31.0	Runs the YOLOv8 neural network on the laptop CPU
YOLOv8-nano (Ultralytics, COCO pre-trained)	v8.4.0 weights	Recognising 80 everyday object classes (person, bottle, chair …)
Ultralytics, PyTorch, torchvision	—, 2.11, 0.26	One-time conversion of the YOLOv8 weights to the ONNX format
Depth Anything V2 (metric, ViT-S)	pre-trained	Experiment: depth from a single image (not used for driving)
OpenCV (headless)	5.0.0	Image handling in the depth experiment
unittest, unittest.mock	standard library	97 automated tests with simulated car, sonar and detector
HTML5, CSS3, JavaScript	—	Dashboard and VR View user interfaces (no framework)
SVG	—	Speed gauge, minimap, path maps, detection boxes
Web Speech API	browser	Speech recognition (voice in) and speech synthesis (car speaks)
Fullscreen API, Page Visibility API	browser	Immersive VR View; safety stop when the tab is hidden

Technology	Version	Where and why it is used
IP Webcam (Android)	—	Streaming the phone camera as MJPEG over HTTP
nRF Connect for Mobile (Android)	—	Discovering and testing the HC-08 BLE service
ReportLab	4.4.9	Producing this report
AI coding assistant (Claude Code)	—	Assisted with writing, testing and debugging code under the team's direction

4.1 Why these choices

- **Laptop as the brain:** the Uno has only 2 KB of RAM; images, speech and neural networks must run elsewhere.
- **Phone as the camera:** better optics and Wi-Fi streaming than an Arduino camera module, at no extra cost.
- **Plain Python and JavaScript, no frameworks:** nothing to install on the client, easy to read and to explain.
- **Regular-expression grammar instead of an NLP library (e.g. NLTK):** the car understands a small, fixed command set; a strict grammar is offline, instant, fully testable and never "guesses" — an unknown phrase never moves the car.
- **ONNX Runtime instead of PyTorch at run time:** a small (≈ 15 MB) inference engine that runs the network fast on a CPU.

5 Firmware and Communication

The firmware keeps the original car code — motor functions, remote control, line tracking and obstacle avoidance — and adds a text protocol through which the laptop requests short, safe movements. It uses 15,760 bytes of flash (48 %) and 1,066 bytes of RAM.

5.1 Bounded pulses

The central safety idea is that the Arduino, not the laptop, ends every movement. When a pulse is requested it records the time at which the pulse must stop and stops the motors itself, so the car cannot keep driving if the laptop crashes or the Bluetooth link drops.

```
// start of a pulse (1-350 ms requested by the laptop)
aiMoveActive = true;
aiMoveStopAt = millis() + pulseMs;

// checked on every pass of loop()
if (aiMoveActive && (long)(millis() - aiMoveStopAt) >= 0) {
  stop();                       // ENA = ENB = LOW
  clearAiMovement();
  Serial.println("AI:MOVE_DONE");
}
```

Listing 5.1 — Self-ending movement pulse (SmartCar_Presentation.ino).

Table 5.1. Laptop-to-Arduino command protocol (excerpt).

Command	Meaning	Reply / note
f b l r s	Original continuous driving and stop	unchanged original behaviour
1 / 2	Line tracking / obstacle avoidance	same as remote keys
P	Presentation mode (“mode 4”) for the laptop	PRESENTATION:READY
@F350 (newline)	Manual pulse F/B/L/R of 1–350 ms	AI:ACK:F … AI:MOVE_DONE
~F350 (newline)	Route pulse with 20 cm front stop	stops by itself if too close
D	Front distance	AI:DIST_CM=42 or AI:DIST:INVALID
G	Servo sweep left / centre / right	AI:SCAN:L=68,C=29,R=35
S	Stop the current pulse immediately	AI:ACK:S
! / C	Emergency stop (latched) / clear	all movement refused while latched
?	Status	STATUS:MODE=PRESENTATION,MOTION=0,EMERGENCY=0

5.2 Bluetooth Low Energy link

The module was sold as “HC-01” but advertised itself as HC-08. Normal Windows pairing failed, so the Android app nRF Connect was used to inspect it: it exposed a UART service (FFE0) whose characteristic FFE1 supports notify, read and write-without-response, and notifications carried text from the Arduino. The laptop then connected directly with Bleak.

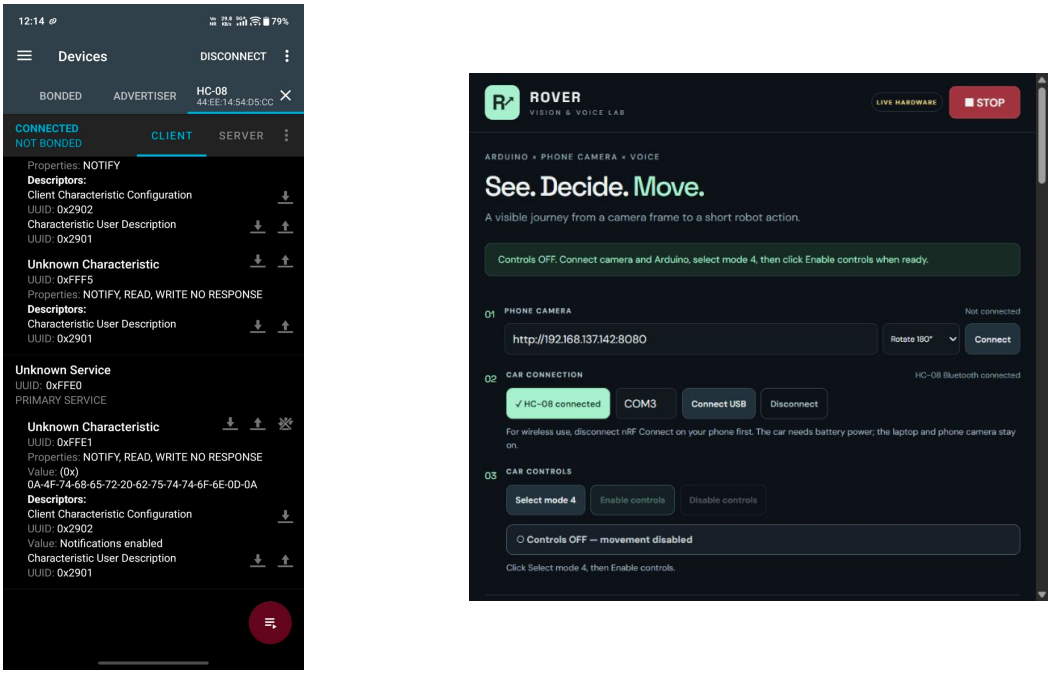


Figure 5.1. *Proof of the Bluetooth link. Left: nRF Connect showing the HC-08 service and its notify/write characteristic. Right: the ROVER dashboard reporting "HC-08 connected".*

5.3 Camera link

IP Webcam serves an MJPEG stream at `http://<phone-ip>:8080/videofeed`. A background thread in phone_camera.py keeps only the newest frame, and frames older than two seconds are rejected so that decisions are never based on an old picture.

6 Features — Working, Technology and Use Cases

Each feature below is described in the same way: what it does, how it works, the technology behind it, a real-world use case, and the evidence that it works. Table 6.1 first summarises which technologies each feature depends on.

Table 6.1. Feature-to-technology matrix.

Feature	Hardware	Software and algorithms
Remote-control modes	IR receiver, ultrasonic, servo, line sensors	Arduino C++, IRremote, Servo
Dashboard and manual driving	HC-08 / USB	Python http.server, HTML/JS, bounded pulses
Voice control	laptop microphone	Web Speech API, regex grammar (re)
Routes by text or voice	—	route parser, timing model (70 ms/cm, 12 ms/°)
Camera floor analysis	phone camera	Pillow, NumPy, floor-colour continuity
Ultrasonic safety gates	HC-SR04, SG90	firmware gates, laptop clearance checks
Pass obstacle	camera + ultrasonic	dead reckoning, avoid-and-rejoin logic, SVG path map
VR View	camera + ultrasonic	Fullscreen API, SVG HUD, danger-level formula
Speed levels 1–10	—	pulse length + pause scheduling
Analyze and report	ultrasonic sweep + camera	rule-based report generator
Autopilot with corrections	camera + ultrasonic	obstacle memory, goal steering, shared control
Teach-and-repeat, go home	—	path recording, Ramer–Douglas–Peucker simplification, autopilot legs
Go to object	camera + ultrasonic	YOLOv8n via ONNX Runtime, visual servoing
Follow me	camera + ultrasonic	YOLOv8n person detection, distance keeping
What do you see	camera	YOLOv8n + speech synthesis
Safety architecture	Arduino + laptop	bounded pulses, epochs, heartbeat, process isolation

6.1 Remote-control modes (original car)

What it does. Drives the car with the IR remote and runs two autonomous modes entirely on the Arduino: obstacle avoidance and line tracking.

How it works. In obstacle avoidance the car drives forward until the front distance is 40 cm or less, then sweeps the servo to measure left and right and turns toward the more open side. In line tracking the three IR sensors steer the car to keep a dark line under its centre.

Technology	Arduino C++, IRremote 4.7.1 (NEC codes), Servo library, HC-SR04, line sensors.
Real-world use	Educational robots; line-following carts in factories and hospitals.
Evidence	Remote key UP was captured over USB as code `0xB946FF00` and decoded correctly; modes kept working after all new features were added.

6.2 Web dashboard and manual driving

What it does. A local web page shows the camera, sensor readings, the proposed action and a timeline of every command sent and confirmed. Buttons drive the car in short steps.

How it works. The browser sends JSON requests to the Python server on 127.0.0.1:8765. Controls must be explicitly enabled; each click becomes one bounded pulse (350 ms forward/back, 120 ms turn), and the timeline separates “sent”, “acknowledged” and “completed”.

Technology	Python http.server, threading, HTML5/CSS3/JavaScript, Fetch API, pySerial / Bleak.
Real-world use	Remote operation consoles for delivery and inspection robots.
Evidence	Figure 6.1 shows the dashboard; the command timeline records every ACK and MOVE_DONE from the Arduino.

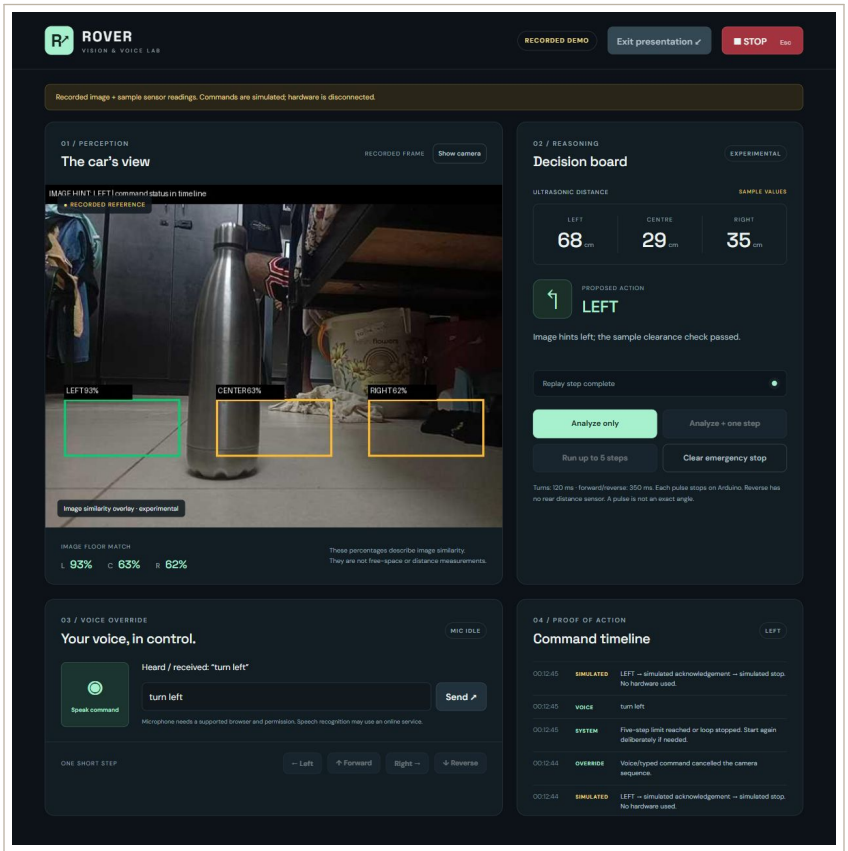


Figure 6.1. *ROVER dashboard (recorded-demo mode): camera with analysis boxes, ultrasonic readings, proposed action, voice panel and command timeline.*

6.3 Voice control

What it does. The operator speaks commands such as “forward”, “turn right 30 degrees and move forward”, “speed level 3” or “stop”.

How it works. The browser’s Web Speech API turns speech into text. Python normalises the text (number words become digits, units are standardised), splits it into clauses on “and/then”, and matches each clause against a strict grammar. Unknown phrases are rejected and nothing moves. “Stop” is acted on from the partial transcript, before the sentence ends, to save time.

Technology	Web Speech API (en-IN), Python regular expressions, unit tests for every phrase pattern.
Real-world use	Hands-free control for people with limited hand movement, operators wearing gloves, assistive robots.
Evidence	The parser is covered by dedicated unit tests (e.g. “turn left forty five degrees” → turn LEFT 45; “speed 7.6” → level 8).

```
"Turn right thirty degrees and move forward"
  normalise -> "turn right 30 deg and move forward"
  split     -> ["turn right 30 deg", "move forward"]
  actions   -> [("turn", "RIGHT", 30), ("move", "FORWARD", None)]
```

Listing 6.1 — How one spoken sentence becomes actions.

6.4 Routes by text or voice

What it does. A route such as “forward 30 cm, then right 30 degrees, then forward 20 cm, then stop” is previewed and then executed.

How it works. The route parser converts each step into a timed sequence of bounded pulses using the calibration 70 ms per cm and 12 ms per degree. A route must be previewed exactly before it can run.

Technology	Python route parser (re), timing model, bounded pulses.
Real-world use	Fixed-path delivery in warehouses, hospitals and campuses.
Evidence	Figure 6.2 shows the route panel; parser tests cover commas, “then”, “again” and invalid clauses.

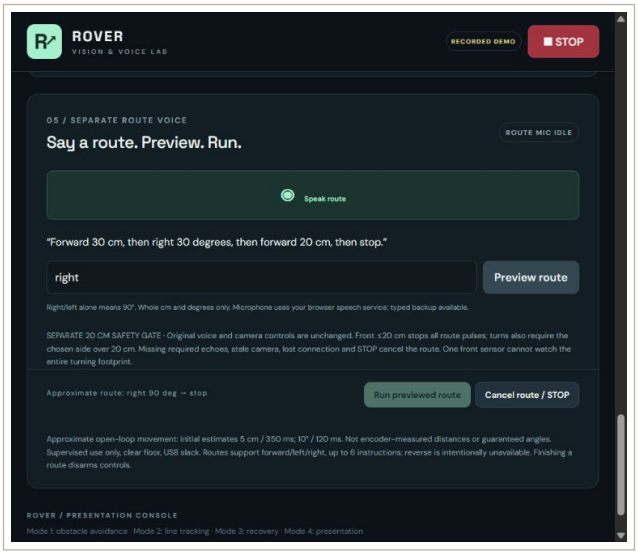


Figure 6.2. *Route panel: a spoken or typed route is previewed before it can be run.*

6.5 Camera floor analysis

What it does. Shows, for the left, centre and right of the picture, whether the floor ahead looks clear, as green (clear) or red (blocked) boxes on the live video.

How it works. Floor-colour continuity: the colour of the floor just in front of the car is compared with the area a little further ahead (chapter 7). The bottom 20 % of the frame is cropped so the car’s own sensor is not analysed.

Technology	Pillow, NumPy, median statistics, RGB Euclidean distance, thresholding.
Real-world use	Robot vacuum cleaners and floor robots; low-cost vision for education.
Evidence	Figures 7.2–7.3: real frames where a bottle turns the matching zone red while empty floor stays green.

6.6 Ultrasonic safety gates

What it does. Prevents unsafe camera-driven moves and measures distances for every decision.

How it works. Camera mode allows forward only if the front is more than 55 cm, and a turn only if that side is more than 45 cm. In manual mode readings are shown as warnings only, so the operator stays in charge.

Technology	HC-SR04, SG90 servo sweep, firmware and laptop clearance checks.
Real-world use	Parking sensors in cars; collision warnings on automated guided vehicles.
Evidence	Logged decisions such as “Front clearance is 55 cm or less; forward step blocked.”

6.7 Pass obstacle

What it does. “Forward 200 cm” drives to a point straight ahead, going around anything in the way and rejoining the original line.

How it works. A stop-check-move loop: blocked ahead → turn 10° toward the clearer side; once 20 cm past the obstacle along the line, steer back toward the destination; finish facing the original direction. The position is estimated by dead reckoning and drawn as a path map.

Technology	Dead reckoning, camera + ultrasonic fusion, SVG path map.
Real-world use	Delivery robots and AGVs that must bypass a box in an aisle and continue.
Evidence	Real run log: “Blocked: ultrasonic 45 cm [L85 C99 R48 %] → turn left”; behaviour later refined after the logs showed the sonar cone problem (section 9.4).

6.8 VR View — immersive driving interface

What it does. A full-screen, game-style display: live camera, directional red danger glows, heading compass, speed gauge, path minimap, command overlay and report panel.

How it works. The browser polls the server four times a second. Glow strength = (80 − distance) ÷ 65, limited to 0–1, plus a contribution from camera zones that look blocked; it pulses when critical. The rear shows “NO SENSOR” honestly.

Technology	Fullscreen API, SVG, CSS animation, Page Visibility API, Web Speech API.
Real-world use	Telepresence and remote inspection in hazardous places (pipes, collapsed buildings, mines); drone pilot displays.
Evidence	Figure 6.3 shows the interface during a mission; layout verified at laptop resolution with no browser errors.

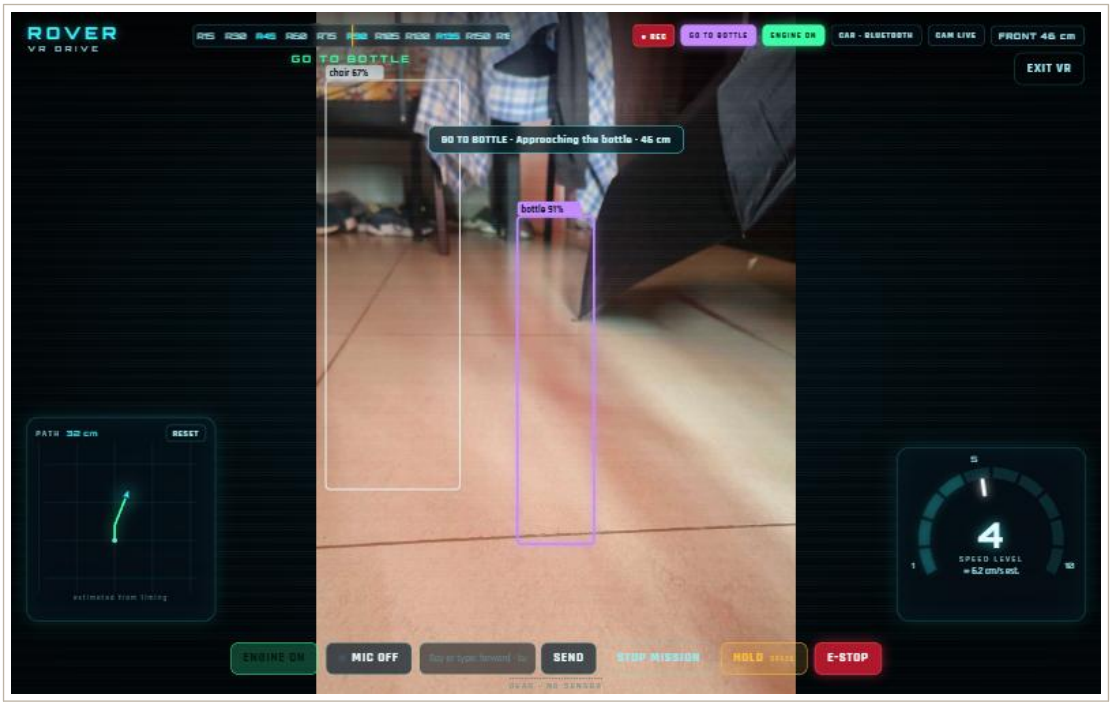


Figure 6.3. VR View during a "go to" mission over the live camera feed (detection boxes and readings shown with sample values for illustration).

6.9 Speed levels 1–10

What it does. The operator chooses a speed level by voice; any number is rounded to the nearest level within 1–10 (7.6 → 8).

How it works. Motor power is unchanged; the level changes the pulse length (170–350 ms) and the pause between pulses, so the average speed really changes without a firmware update (Figure 6.4).

Technology	Pulse scheduling in vr.py, SVG speed gauge.
Real-world use	Speed zones in factories and hospitals: slow near people, faster in empty corridors.
Evidence	Speeds computed from the calibration: ≈ 2.1 cm/s at level 1 to ≈ 14.3 cm/s at level 10.

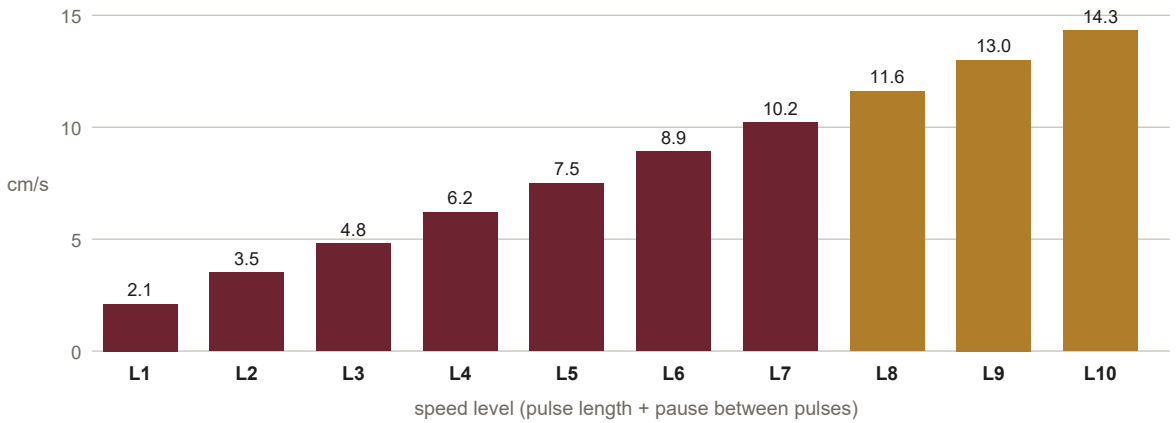


Figure 6.4. Estimated average speed for each level (from the timing calibration).

6.10 Analyze and report

What it does. “Analyze and report” produces a plain-language summary of the surroundings with a recommended direction.

How it works. The servo sweeps the ultrasonic sensor left, centre and right; the camera zones are added; rules turn the readings into sentences such as “Obstacle about 40 cm ahead. Left looks most open. Recommended: slow down.”

Technology	Servo sweep, camera zones, rule-based report generator.
Real-world use	Inspection robots that write a quick status for an operator log.
Evidence	Report rules are unit-tested (e.g. obstacle at 25 cm on the right → “left” recommended).

6.11 Autopilot with operator corrections (shared control)

What it does. “Autopilot 200 cm” drives to a destination by itself; the operator can correct it at any moment by voice (“dodge left”, “hold”, “resume”).

How it works. Each step stops, looks and decides (Figure 7.7). The autopilot remembers where it saw obstacles and never plans a path closer than 22 cm to one — even after the narrow sonar cone loses sight of it. An operator correction is kept for about 30 cm, then the car rejoins its line. If it would need to turn more than 60°, it pauses with “NEEDS HELP” and continues after the operator steers.

Technology	Dead reckoning, obstacle memory, goal steering, sensor fusion, human-in-the-loop control.
Real-world use	Mars-rover-style supervised autonomy; inspection and bomb-disposal robots; one operator supervising several robots.
Evidence	Simulated test: a bottle 40 cm ahead is passed at ≈ 26 cm, the car goes at most 35 cm to the side and finishes 0.7 cm from the line (Figure 9.1).

6.12 Teach-and-repeat and Go home

What it does. “Record route”, drive, “stop recording” saves a route; “repeat route” drives it again from the start; “go home” retraces the path to the starting point.

How it works. The driven path is converted to coordinates relative to the start pose and simplified with the Ramer–Douglas–Peucker algorithm, which keeps the corners. Each segment is then driven as an autopilot leg, so obstacles are still avoided. The saved route survives restarts (JSON file).

Technology	Path recording, Ramer–Douglas–Peucker simplification, coordinate transforms, autopilot legs.
Real-world use	Patrol and security robots, cleaning robots returning to their dock, repeated delivery runs.
Evidence	Simulated test: an L-shaped route (40 cm, 90° turn, 30 cm) is repeated to within 8 cm of its end point and “go home” returns to within 8 cm of the start, facing the start direction.

6.13 Go to object

What it does. “Go to the umbrella”, “find my bag” or “come to me”: the car searches, turns toward the object, approaches it and stops about 25 cm away, then says “I reached the …”.

How it works. Visual servoing: the horizontal position of the detected box sets the turn angle (the camera sees about 50° across); when centred the car moves forward up to 15 cm, stopping when the ultrasonic reads 25 cm or the object fills 75 % of the picture height. If the object is not visible it turns 30° at a time, giving up after a full circle.

Technology	YOLOv8-nano (COCO) via ONNX Runtime, ultrasonic, speech synthesis.
Real-world use	Fetching and assistive robots for elderly or visually impaired people; warehouse picking.
Evidence	Simulated test reaches the object within the stop distance; real detection results in section 7.2.

6.14 Follow me

What it does. The car follows a person, keeping them centred and 50–90 cm away; it looks around if it loses them and says “I lost you” after a while.

How it works. The tallest detected person is treated as the nearest; turning keeps them within ±20 % of the picture centre; the ultrasonic (or the box height when there is no echo) keeps the distance. Steering or “hold” ends the mission.

Technology	YOLOv8n person class, ultrasonic distance keeping.
Real-world use	Shopping-trolley and luggage-carrying robots; carrying equipment in hospitals.
Evidence	Simulated test: the car catches up, holds the distance band and turns to follow a person walking to the side.

6.15 What do you see — spoken scene description

What it does. The car names the objects in view and where they are, out loud.

How it works. One detection pass; objects are grouped as left / ahead / right and the front distance is added for objects straight ahead. The browser speaks the sentence and pauses the microphone so the car does not hear itself.

Technology	YOLOv8n, Web Speech API speech synthesis.
Real-world use	Assistive description for visually impaired users; voice-first robot interfaces.
Evidence	Real session log: “Car says: I see a person on the right.”

6.16 Safety architecture

What it does. Keeps the car safe when software, people or communication fail.

How it works. Five layers of protection (Figure 6.5). The newest layer runs object detection in a separate process: during testing the native inference library crashed the server; after isolation a crash only restarts the helper.

Technology	Bounded pulses, emergency latch, epoch counter, heartbeat, watchdog, process isolation, faulthandler crash log.
Real-world use	Fail-safe design required for any machine operating near people.
Evidence	Detector helper deliberately killed during a test — the next request restarted it automatically and the server stayed up.

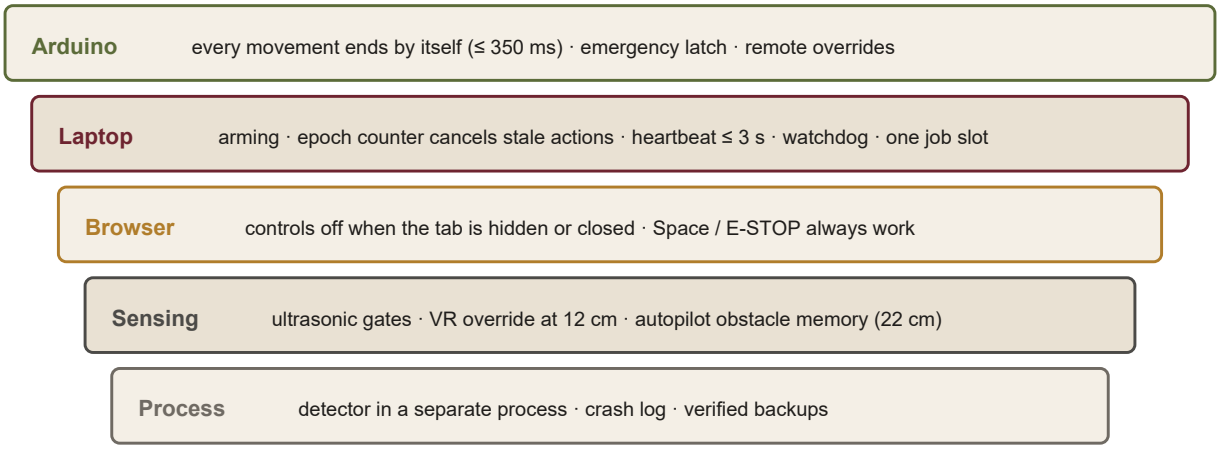


Figure 6.5. *Layers of protection, from the Arduino to the development process.*

7 Computer Vision and Object Detection

7.1 Floor-colour continuity (image processing)

The camera does not measure distance. Instead each frame answers one question per direction: does this part of the picture still look like the floor just in front of the car? The method needs no training data, runs in milliseconds, and every decision can be explained with numbers shown on screen.

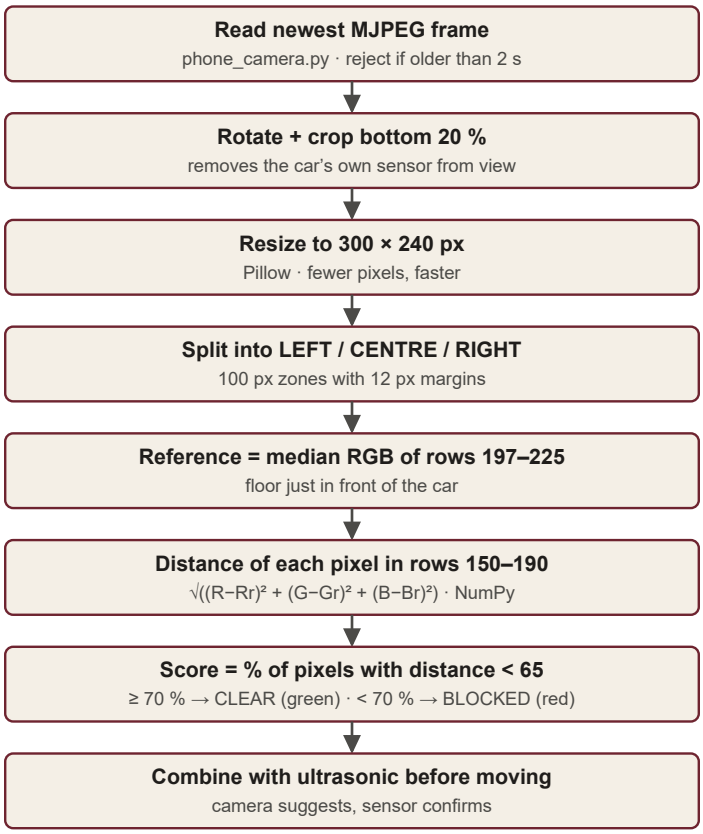


Figure 7.1. *Floor-colour continuity pipeline.*

```
rgb = np.asarray(image.convert("RGB").resize((300, 240)), dtype=np.float32)
for index, zone in enumerate(("LEFT", "CENTER", "RIGHT")):
    x0, x1 = index * 100 + 12, (index + 1) * 100 - 12
    near  = rgb[197:225, x0:x1]                    # floor just in front
    probe = rgb[150:190, x0:x1]                    # a little further ahead
    reference = np.median(near.reshape(-1, 3), axis=0)
    distance = np.linalg.norm(probe - reference, axis=2)
    scores[zone] = round(float(np.mean(distance < 65) * 100))
```

Listing 7.1 — Core of the floor analysis (vision_preview.py).

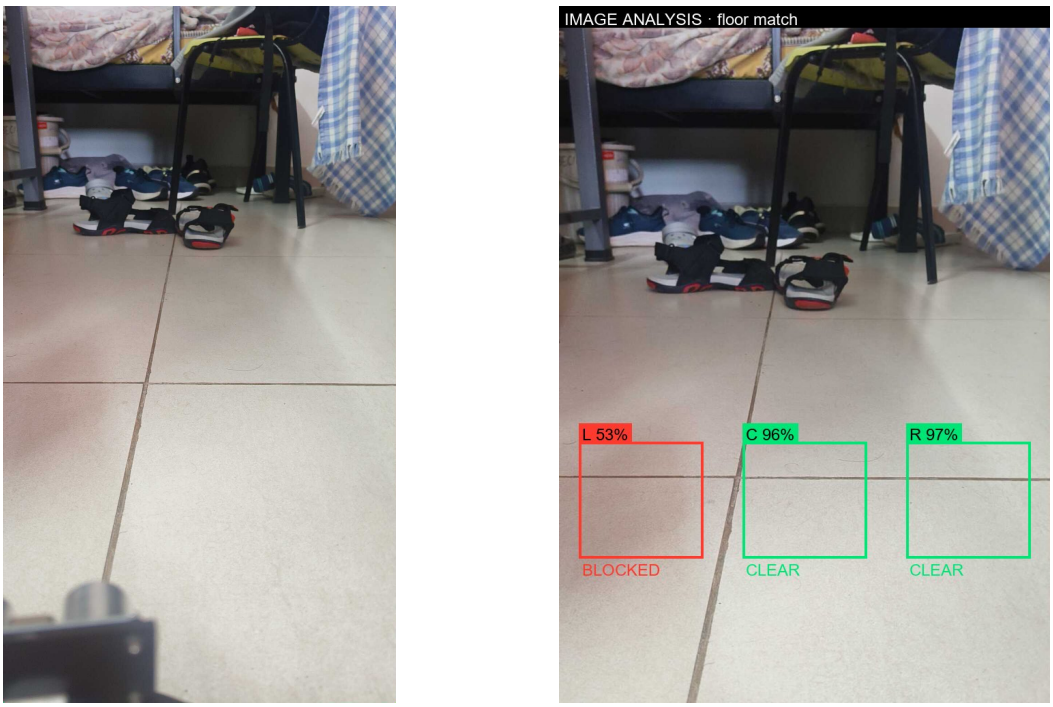


Figure 7.2. *Real frames. Left: raw camera frame — the car's own ultrasonic sensor is visible at the bottom. Right: after the 20 % crop with live analysis; CENTRE 96 % and RIGHT 97 % are clear, LEFT 53 % is a false block caused by a reddish light patch.*



Figure 7.3. *Real frames with a bottle. Left: bottle on the right lowers the RIGHT score to 65 %. Right: bottle on the left gives LEFT 69 % BLOCKED while centre and right stay clear (ultrasonic 43 cm).*

7.2 Object detection with YOLOv8

For object finding and following, the project uses **YOLOv8-nano**, a convolutional neural network pre-trained by Ultralytics on the COCO dataset (80 everyday classes). The official weights were converted once to the ONNX format and are executed with ONNX Runtime on the laptop CPU. The pre- and post-processing (letterboxing, decoding the 84 × 8400 output, non-maximum suppression)

was written in NumPy.

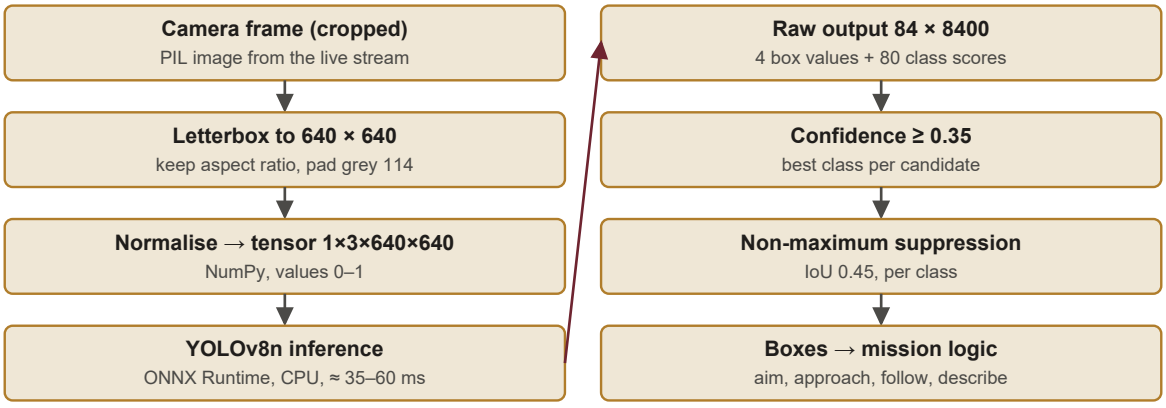


Figure 7.4. Object detection pipeline.

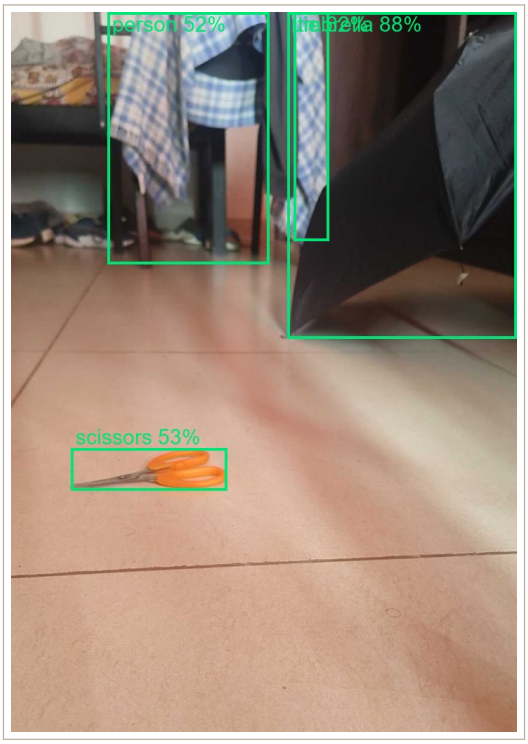


Figure 7.5. Real detection on the live car camera: umbrella 88 % and scissors 53 % are correct; "person 52 %" is a false detection of a shirt hanging on a chair.

Table 7.1. Measured detector performance on the development laptop.

Model	Input size	Time per frame	Live camera result	Steel flask recognised?
YOLOv8-nano	320 px	≈ 15–25 ms	objects too small in portrait frames	no
YOLOv8-nano (used)	640 px	≈ 35–60 ms	umbrella 88–91 %, scissors 53 %, person	no (at best “vase” 12 %)
YOLOv8-small	640 px	≈ 80–105 ms	umbrella 90 %, weak “bottle” 26 %	no

The measurements show an important, honest limitation: neither pre-trained model recognises the team’s stainless-steel flask as a bottle, because COCO mostly contains plastic and glass bottles photographed from human eye height. Common objects and people are detected well. The proper fix — fine-tuning the detector on photographs taken by the car itself — is described in chapter 10.

7.3 Depth estimation experiment

To test whether a neural network could give real distances from one camera image, the pre-trained **Depth Anything V2** model (metric version, ViT-Small) was run with PyTorch on the laptop. It ordered objects correctly but estimated a bottle at ≈ 1.5 m when it was ≈ 80–90 cm away, so it was not used for driving; the ultrasonic sensor remains the only distance source.



Figure 7.6. *Depth Anything V2 on a car-camera photograph (red = near, blue = far). Yellow dots mark where the bottle’s depth was sampled.*

7.4 Autopilot decision logic

The autopilot, pass-obstacle mode and the legs of teach-and-repeat all use the decision loop in Figure 7.7.

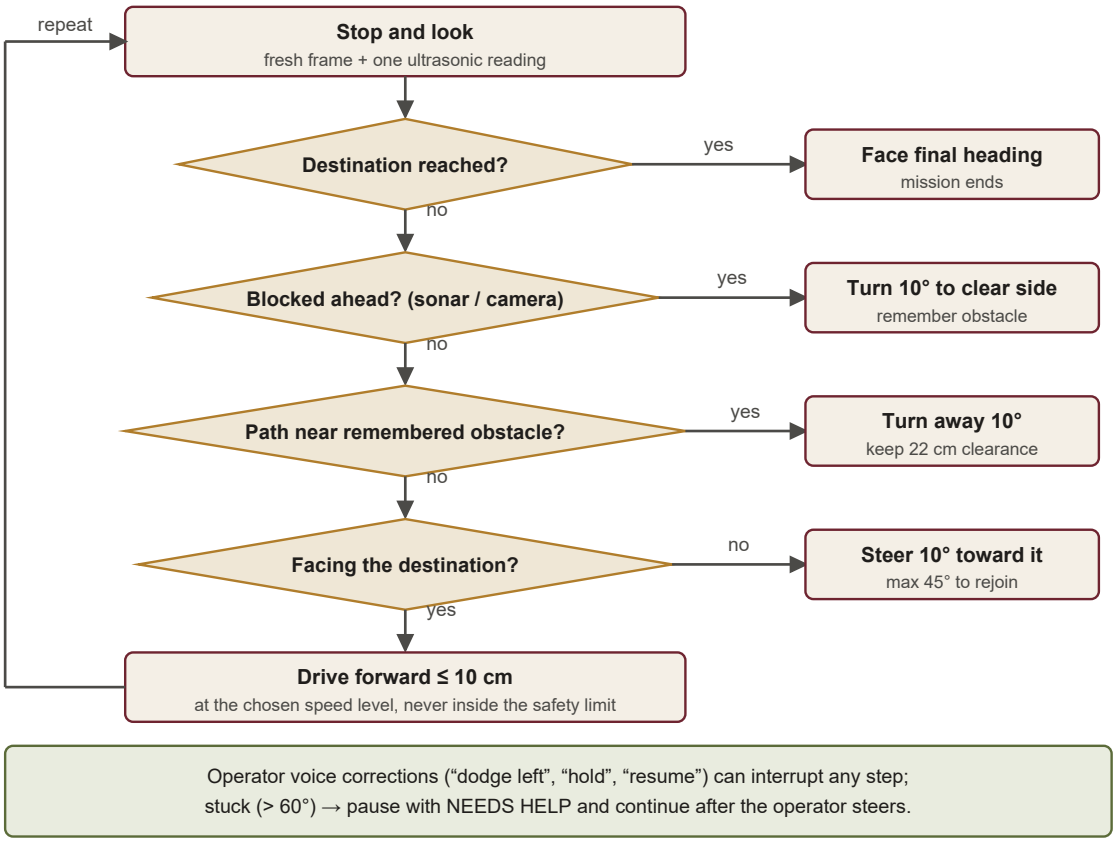


Figure 7.7. *Autopilot decision loop with operator corrections.*

8 Motion Model and Calibration

Because the car has no wheel encoders or compass, its movement is estimated from time. The constants come from floor observations: about 5 cm in a 350 ms forward pulse and about 10° in a 120 ms turn.

Table 8.1. Timing calibration.

Quantity	Value	Meaning
Forward / reverse	70 ms per cm	350 ms ≈ 5 cm at PWM 200
Turning	12 ms per degree	120 ms ≈ 10°
Pulse limit	1–350 ms	enforced by both the laptop and the Arduino
Heading range	−180° to 180°	kept normalised so the compass and maths stay correct

```
distance_cm = pulse_ms / 70
east   += distance_cm * sin(heading)
north  += distance_cm * cos(heading)
heading = normalise(heading + turn_ms / 12)      # right positive
```

Listing 8.1 — Dead-reckoning equations used for the path map, autopilot and missions.

Errors accumulate: battery level, floor grip and the phone's weight change the real movement. Very short pulses (under about 80 ms) barely move the car because of motor start-up, which is why turns use steps of at least 10°. Fitting a gyroscope and wheel encoders would remove most of this error.

9 Testing and Results

9.1 Automated tests

All software logic is covered by automated tests written with Python's unittest. Tests that involve driving use a *simulated car* that moves according to the timing model, a *simulated sonar* that only sees an object when the car points at it, and a *simulated detector* that returns a box only when an object is inside the camera's field of view. This allowed the autonomous behaviour to be checked safely before any real driving.

Table 9.1. Automated test suite.

Test file	Tests	What is checked
test_dashboard.py	29	decision rules, camera and route safety gates, route parser, pass-obstacle mode, voice safety
test_ble.py	11	Bluetooth packet framing, bounded packets, firmware verification
test_vr.py	15	voice grammar, speed rounding, danger levels, VR session, autopilot (bottle, corrections, help, hold)
test_missions.py	15	record / repeat / go home, go to object, follow me, scene description, missing model
smartcar_laptop (14 files)	27	camera, command manager, vision preview, probes
Total	**97**	**all passing**

9.2 Simulated autopilot result

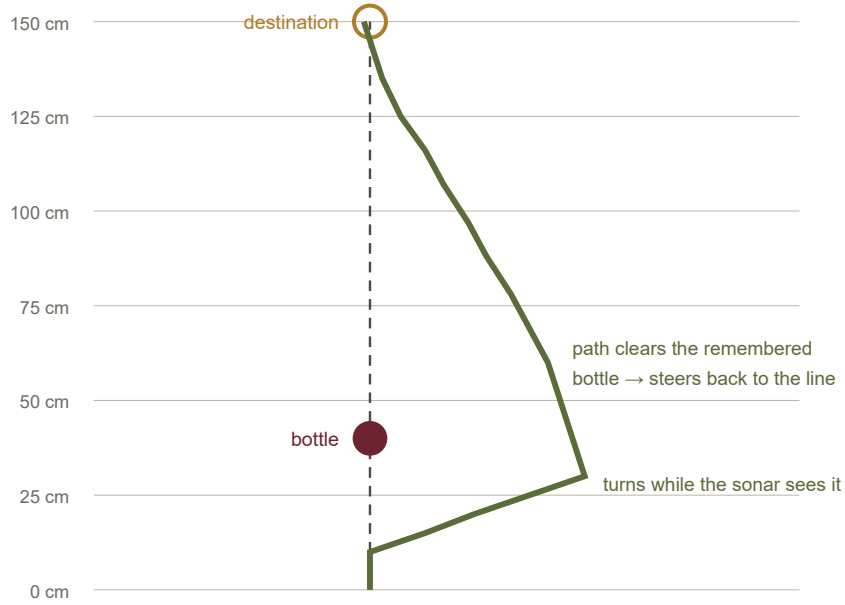


Figure 9.1. *Real output of the simulated autopilot test (bottle 40 cm ahead, destination 150 cm): closest approach ≈ 26 cm, widest deviation 35 cm, final error 0.7 cm.*

9.3 Real-world evidence

- Firmware start-up reports the capabilities `CAPS:MANUAL_ADVISORY_V1`, `PRESENTATION_V1` and `ROUTE_V1_20CM`.
- Remote key UP captured as `0xB946FF00`; Bluetooth service discovered with nRF Connect (Figure 5.1).
- Live floor analysis and bottle tests (Figures 7.2–7.3); live object detection (Figure 7.5).
- Pass-obstacle log: *"Blocked: ultrasonic 45 cm [L85 C99 R48 %] → turn left"*.
- VR session log: *"Car says: I see a person on the right."*

9.4 Problems found and solved

Table 9.2. Debugging history.

Problem	Cause	Solution
Car did not move at all	Discharged battery; Arduino still lit from USB	Battery charged; motor-only test program separated hardware from software
Uploads failed ("not in sync")	HC-08 shares the USB serial pins	Remove the module (power off) before uploading
Voice command caused an emergency stop	Hidden second dashboard tab disabled controls	Extra tab removed; a command with controls off now shows a message
Pass mode kept turning to 75°	Sonar cone kept seeing the bottle beside the car	Camera decides; ultrasonic limit 30 cm; 10° turns
Turned back into the obstacle	Returned after a fixed distance	Return only when 20 cm past the obstacle along the line
Autopilot passed 10 cm from the bottle	Sonar loses sight after a small turn	Obstacle memory with 22 cm clearance
Combined voice command lost its first part	Second action cleared the queue	Queue cleared once per command
Bottle too small for the detector	320 px input on a 1080 × 1920 frame	Model re-exported at 640 px
Server crashed during detection	Native crash in the inference library	Detection moved to a separate helper process; crash log added

10 Limitations and Future Work

- **Position from timing only.** An MPU-6050 gyroscope would give accurate turns (it needs the I²C pins A4/A5, which the ultrasonic uses today, so rewiring is required) and wheel encoders would give real distance and speed.
- **Front-only sensing.** Nothing sees behind the car; a rear sensor would make the danger display truly 360°.
- **Simple floor analysis.** Shadows, light patches and patterned floors can cause false blocks; the ultrasonic check and the operator reduce the risk.
- **Detector does not know every object.** Pre-trained YOLOv8 missed the steel flask and mistook a hanging shirt for a person.
- **Voice latency** of about 1–2 seconds; manual stop controls must stay ready.
- **Speed by scheduling.** True speed control would add a firmware command that sets PWM per pulse.

10.1 Training plan for a custom detector

The reliable way to recognise the project's own objects is to fine-tune YOLOv8 on photographs taken by the car:

1. Collect 60–100+ frames per object from the car camera in different positions, distances and lighting.
2. Draw a bounding box around the object in each frame (labelling tool) and split into training, validation and test sets (e.g. 70/20/10).
3. Fine-tune the pre-trained YOLOv8-nano weights for a short run (transfer learning).
4. Evaluate precision, recall and mAP on the test set and check the speed on the laptop.
5. Export to ONNX and replace the model file — the rest of the software stays unchanged.

11 Conclusion

ROVER shows that a very inexpensive robot car can offer natural voice control, an immersive camera interface, obstacle-aware driving, object recognition and supervised autonomy when a smartphone and a laptop take on the roles of eyes and brain. The main contribution is not a new algorithm but a careful integration: transparent image analysis combined with real distance sensing, a pre-trained neural network used through a lightweight runtime, an autopilot that accepts the operator's spoken corrections, and a safety design in which every movement ends by itself.

The system was verified with 97 automated tests and documented with real photographs, logs and measurements, including the cases where it falls short. These limitations point directly to the next steps: position sensors for accurate navigation and a detector fine-tuned on the car's own images.

References

[1] Arduino. *Arduino Uno Rev3 — Documentation.* https://docs.arduino.cc/hardware/uno-rev3

[2] Arduino-IRremote contributors. *IRremote library.* https://github.com/Arduino-IRremote/Arduino-IRremote

[3] STMicroelectronics. *L298 Dual Full-Bridge Driver — Datasheet.*

[4] Bleak contributors. *Bleak: Bluetooth Low Energy platform-agnostic client.* https://bleak.readthedocs.io

[5] Jocher, G., Chaurasia, A., Qiu, J. (2023). *Ultralytics YOLOv8.* https://github.com/ultralytics/ultralytics

[6] Lin, T.-Y. et al. (2014). Microsoft COCO: Common Objects in Context. *European Conference on Computer Vision (ECCV).*

[7] Microsoft. *ONNX Runtime.* https://onnxruntime.ai

[8] Yang, L. et al. (2024). Depth Anything V2. *arXiv:2406.09414.*

[9] W3C Web Platform Incubator Community Group. *Web Speech API.* https://webaudio.github.io/web-speech-api/

[10] Douglas, D. H., Peucker, T. K. (1973). Algorithms for the reduction of the number of points required to represent a digitized line. *Cartographica*, 10(2).

[11] Harris, C. R. et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.

Appendix A — Voice Command Reference

Table A.1. Voice commands understood by VR View.

Say	Effect
forward · reverse 20 cm · stop · hold	drive continuously, a fixed distance, or stop
turn right 30 degrees and move forward	combined commands
speed level 3 · maximum speed · slower	speed levels (rounded to 1–10)
analyze and report	environment report
autopilot 200 cm · dodge left · resume · cancel autopilot	autopilot and corrections
record route · stop recording · repeat route · go home	teach-and-repeat and return home
go to the umbrella · find my bag · come to me	object finding
follow me · stop following	person following
what do you see	spoken scene description
emergency	latched emergency stop

Appendix B — Project Files and Running the System

Table B.1. Main project files.

Item	Location
Firmware	work/SmartCar_Presentation/SmartCar_Presentation.ino
Server and VR logic	work/smartcar_dashboard/server.py, vr.py, missions.py
Object detection	work/smartcar_dashboard/detector.py, detector_worker.py; model work/models/yolov8n.onnx
Image analysis	work/smartcar_laptop/vision_preview.py
Web interface	work/smartcar_dashboard/static/ (index, app, vr)
Tests	work/smartcar_dashboard/test_*.py, work/smartcar_laptop/test_*.py
Start the system	START-ROVER.cmd, then open http://127.0.0.1:8765 (VR View at /vr) in Edge
Backups	work/rollback/ (verified copies before each major change)